

1. 文档概览

本文说明 OpenViking 通用知识库的技术架构、关键数据结构和端到端处理流程。文档重点围绕三个问题展开：第一，外部资料如何被解析为可检索的知识树；第二，对话过程中的经验如何沉淀为长期记忆；第三，AI 应用如何基于会话意图、权限边界和多层索引完成精准检索。

设计目标

OpenViking 不是单一问答机器人，而是 AI 应用的知识与上下文基础设施：上层应用可以是客服、投标、售前、交付助手或企业内部 Copilot；底层统一处理知识入库、记忆沉淀、语义索引、检索召回和多租户隔离。

能力域	解决的问题	对应模块
文档处理	把 PDF、Word、网页、代码仓库、媒体等资料转换为结构化知识树	parse、TreeBuilder、ResourceService
语义加工	为目录和文件生成 L0/L1 摘要，并写入向量索引	SemanticQueue、SemanticProcessor、EmbeddingQueue
记忆处理	把会话、工具调用、业务经验沉淀为用户/Agent 长期记忆	Session、SessionCompressor、MemoryExtractor、MemoryDeduplicator
检索增强	依据用户问题和会话上下文，在资源、记忆、技能中分层召回	SearchService、IntentAnalyzer、HierarchicalRetriever、Rerank
治理与隔离	保证账号、用户、Agent、会话边界清晰，支持异步任务与状态跟踪	RequestContext、Auth、TaskTracker、KnowledgeDocumentRegistry

1.1 总体架构

```
Client / AI App
| Local Client, HTTP API, Console, MCP, Bot
v
Service Layer
|-- ResourceService: add_resource, add_skill, wait_processed
|-- SearchService: find, search, grep, glob
|-- SessionService: message, used context, commit, archive
|-- FS/Content/Relation/Pack/Debug services
v
Core Pipelines
|-- Parser -> TreeBuilder -> SemanticQueue -> Vector Index
|-- Session -> Compressor -> MemoryExtractor -> Dedup -> Memory Store
|-- IntentAnalyzer -> HierarchicalRetriever -> Rerank -> MatchedContext
v
Storage
|-- VikingFS / AGFS: L2 原文、目录、.abstract.md、.overview.md
|-- VectorDB: L0/L1/L2 语义索引、score、metadata
|-- Task/Registry: 入库状态、异步任务、处理遥测
```

架构采用“内容存储 + 向量索引”的双层设计。VikingFS/AGFS 保存原始内容、目录树、摘要文件和元数据；向量库保存可检索记录、层级信息和权限字段。这样既保留可审计的知识来源，又能为 AI 应用提供低延迟语义召回。